



Cifra de Vigenère: Implementação e Criptoanálise

Trabalho de Implementação 1 – CIC0201

Ana Luísa Reis Nascente Gabriel de Sousa
Marina Pimentel Moreno Pedro de Paula Campos

1 de setembro de 2026

- ▶ Cifra clássica de **substituição polialfabética**: cada posição pode usar um deslocamento diferente.
- ▶ Uma chave de comprimento L cria um espaço de 26^L possibilidades.
- ▶ Porém, a **repetição cíclica** da chave mantém padrões estatísticos da linguagem.

Objetivo do trabalho

Implementar a cifra do zero e recuperar **tamanho da chave**, **chave** e **texto claro** sem usar bibliotecas criptográficas prontas.

Vigenère

$$C_i = (P_i + K_{i \bmod L}) \bmod 26$$

$$P_i = (C_i - K_{i \bmod L} + 26) \bmod 26$$

A chave é repetida sobre as letras do texto, produzindo uma sequência de cifras de César.

Política adotada

- ▶ somente A–Z e a–z são cifrados;
- ▶ maiúsculas e minúsculas são preservadas;
- ▶ espaços, números, acentos e pontuação não mudam;
- ▶ esses caracteres não avançam a chave.

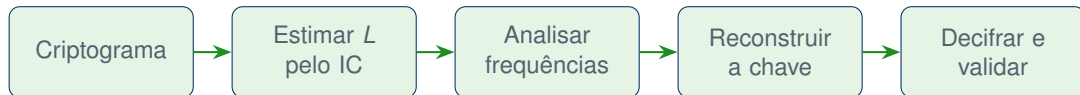
1. Índice de Coincidência

$$IC = \frac{\sum_i n_i(n_i - 1)}{N(N - 1)}$$

- ▶ linguagem natural: aproximadamente 0,065–0,075;
- ▶ distribuição uniforme: aproximadamente $1/26 = 0,038$;
- ▶ o maior IC médio indica períodos prováveis da chave.

2. Análise de frequência

- ▶ divide o criptograma por posição da chave;
- ▶ cada grupo vira uma cifra de César;
- ▶ testa os 26 deslocamentos;
- ▶ compara com frequências de português ou inglês e reconstrói a chave.



Componentes: cifrador/decifrador, estimador, frequências PT/EN e ataque automático.

Saída: vários candidatos ordenados para avaliar legibilidade e períodos equivalentes.

- ▶ Projeto unificado em C++17 e CMake.
- ▶ Testes do cifrador, CLI, preservação de caracteres e entradas inválidas.
- ▶ 500 casos determinísticos de cifrar e decifrar.
- ▶ Testes do estimador com chaves de tamanhos 5, 7, 11 e 16.
- ▶ Teste integrado com texto conhecido em português.

Resultado da suíte

7/7

testes aprovados

Entrada do ataque

Idioma: PT

Limite: 20

Chave desconhecida pelo pipeline

Primeiro candidato

Tamanho: 9

SEGURANCA

- ▶ texto claro recuperado integralmente;
- ▶ espaços, pontuação e parágrafos preservados;
- ▶ resultado igual ao arquivo de referência.

Período equivalente

O segundo candidato foi tamanho 18 e chave

SEGURANCASEGURANCA

Ele também decifra corretamente por repetir duas vezes a chave fundamental.

Decisão: priorizar a menor chave equivalente.

Conclusões

O que o experimento demonstrou



- ▶ A implementação cifra e decifra corretamente, preservando a política de caracteres definida.
- ▶ O pipeline recuperou **tamanho 9**, **chave SEGURANCA** e o **texto claro completo**.
- ▶ Um espaço de chaves grande não garante segurança quando uma chave curta é reutilizada ciclicamente.
- ▶ O desempenho da análise depende do tamanho do texto, do idioma correto e da escolha entre períodos equivalentes.

Conclusão central

A vulnerabilidade explorada está na **repetição da chave**: uma chave aleatória, do tamanho da mensagem e usada uma única vez (One-Time Pad) não apresenta esse mesmo padrão estatístico.